



If we just have to have a winner...

that Jetpack is, in fact a set of APIs, and this is just a part of its function. So, to summarise, we can say that both methods have their own merits and downfalls; but we can safely say that if you were to start today, Jetpack would be easier to understand.

So, having agreed that both sides hold merits, it would only be fair to get on to examining them. First up, the XUL side of the argument:

As we've discussed time and again, a browser written in XUL offers a level of func-

User interface flexibility	XUL overlays offer a great deal of options for building a UI and integrating it into the browser. Using only the SDK's supported APIs you have much more limited options for your UI.
XPCOM	Traditional add-ons have access to a vast amount of Firefox functionality via XPCOM. The SDK's supported APIs expose a relatively small set of this functionality.

tionality and deep integration with the environment that's nigh impossible to match, whatsoever the opposition. This is obviously a huge plus point. Then, with XPCOM, a vast number of possibilities have been exposed and used by developers over the years. XPCOM stands for Cross Platform Component Object Model, and it allowed for a great deal of clarity and consistency in Mozilla's products. Also, it dealt with intricacies throughout the architecture, so the way it empowered a developer was unmatched. But the simple fact of the matter is that most developers don't actually need that kind of power with the tasks that they're doing. If we had to cook up a game analogy, it would be like using a sniper rifle in a head-on battle – powerful, but possibly detrimental, and certainly overkill. This was, undoubtedly, one of the premier motivations for starting the Jetpack project.